

AWS Cloud

A Practical Introduction

Understanding the building blocks behind AWS, hosting and storage to regions, virtual servers and everyday cloud infrastructure.

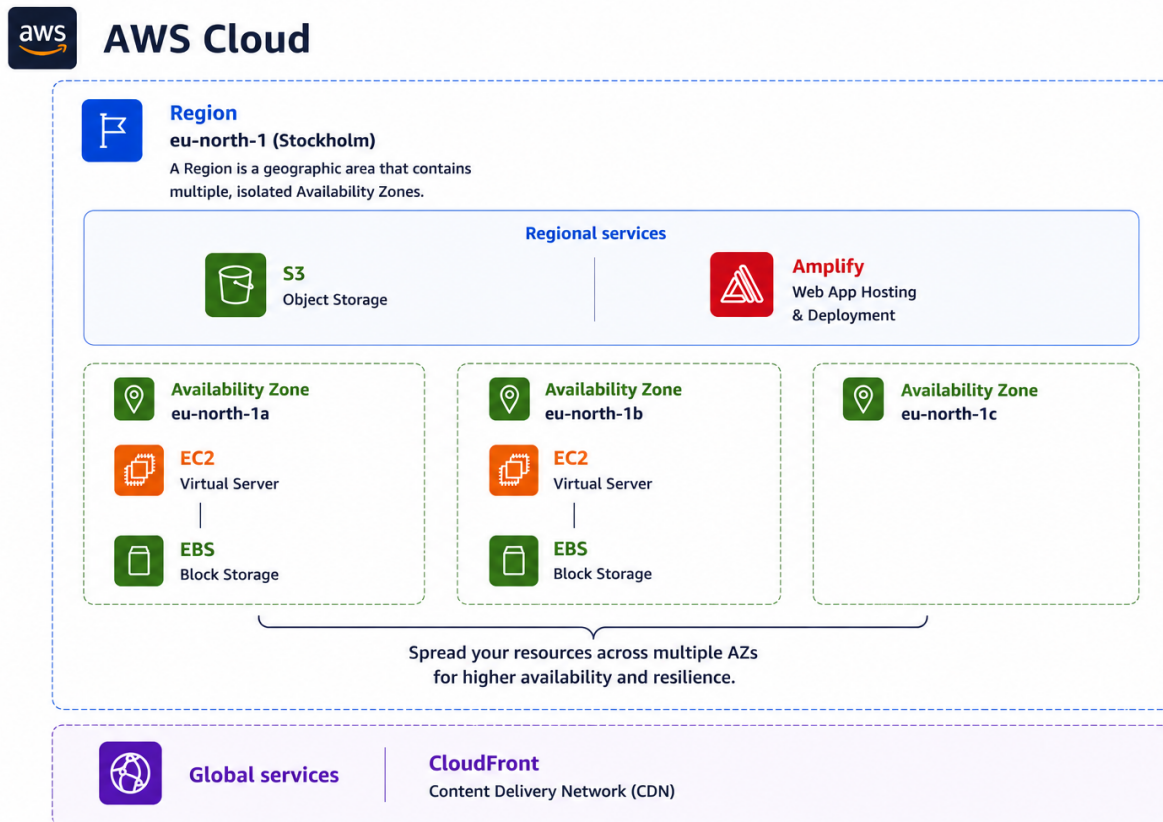
About this guide

This is a practical reference for learning AWS from the ground up. It starts with familiar problems:

- hosting a website
- storing files
- running an application,

we'll gradually introduce the AWS services and infrastructure behind them.

TIPS: Don't memorize the names. For each service, ask: What problem does this solve, where does it run, and what does it connect to?



1. Hosting: Static Files vs Running Code

The first distinction is whether AWS only needs to store and deliver files, or whether a computer must actively execute backend code.

Need	AWS concept	Concrete mental model
Static HTML/CSS/JS/assets	Amazon S3	"Store these files and give them back when requested."
Run a backend process	Amazon EC2	"Give me a computer where I can run my program."
Managed web deployment	AWS Amplify Hosting	"Here is my Git repo; build and deploy it for me."

KEY IDEA

A React/Vite app can still be static hosting.

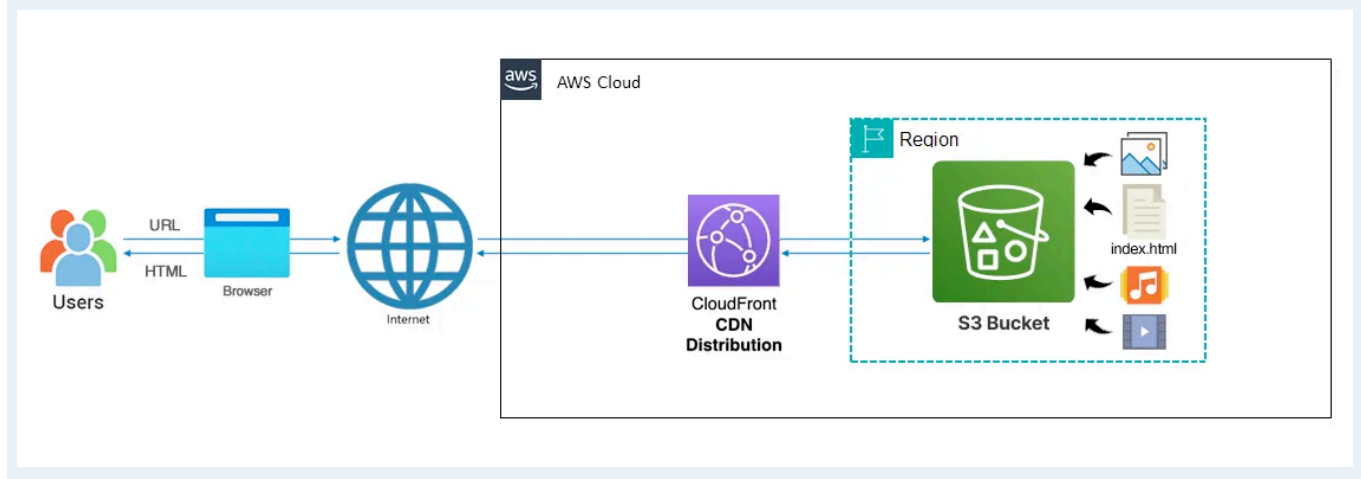
JavaScript runs in the user's browser.

A Spring Boot/FastAPI/Node backend needs server-side compute.

S3 static website hosting

An S3 bucket can serve files such as index.html, app.js, styles.css and images.

For the basic S3 website-hosting exercise, those objects must be publicly readable.



Redirect instead of hosting

The S3 static website settings can also redirect requests rather than serve files.

This is useful when the real website lives somewhere else.

Scenario	Behavior
oldcompany.com to newcompany.com	Old domain redirects visitors to the new domain.
www.example.com to example.com	Only one hostname actually hosts the site; the other redirects.

2. Amazon S3 and Storage Classes

Amazon S3 (Simple Storage Service) is AWS's **object storage service**. It is designed for storing files and other data in the cloud.



Think of it as a huge storage system where your application can put things such as:

- Images and videos
- PDFs and documents
- Backups
- Log files
- Static website files
- Data files used for analytics

How S3 is structured

In S3, you create a **bucket**, and then store **objects** inside it.

For example, an online store might have a bucket called:

```
my-store-images
```

```
products/shoes.jpg  
products/jacket.jpg  
products/hat.jpg
```

The bucket is the container. Each file is stored as an **object** with a unique **key** such as `products/shoes.jpg`.

Different storage classes trade storage cost, access cost, resilience and retrieval speed.

Storage class	Use it when...	Mental shortcut
S3 Standard	Data is accessed frequently.	Normal/default S3.
S3 Intelligent-Tiering	Access patterns are unpredictable.	AWS adjusts tiers based on usage.
S3 Standard-IA	Data is rarely accessed but should remain multi-AZ and quickly retrievable.	Rare, but important.
S3 One Zone-IA	Data is rarely accessed and can be recreated if the AZ is lost.	Rare + recreatable.
Glacier Instant Retrieval	Archived data is rarely accessed but may need immediate retrieval.	Archive, instant access.
Glacier Flexible Retrieval	Archived data can tolerate slower retrieval.	Archive, can wait.
Glacier Deep Archive	Long-term retention where retrieval is very rare.	Put it in the basement.

What does re-creatable data mean?

A useful test is: if this stored copy disappeared tomorrow, could we produce the same data again from another authoritative source?

Example	Re-creatable?	Why?
Generated thumbnail	Yes	Regenerate it from the original image.
Vite dist/ build	Yes	Run the build again from source code.
Encoded 480p video	Yes	Re-encode it from the original video.
User's only uploaded photo	No	There is no source from which to reproduce it.
Only database backup	Usually no	It may be the only recovery copy.

KEY IDEA

Re-creatable does NOT automatically mean One Zone-IA.

The data should also be infrequently accessed.

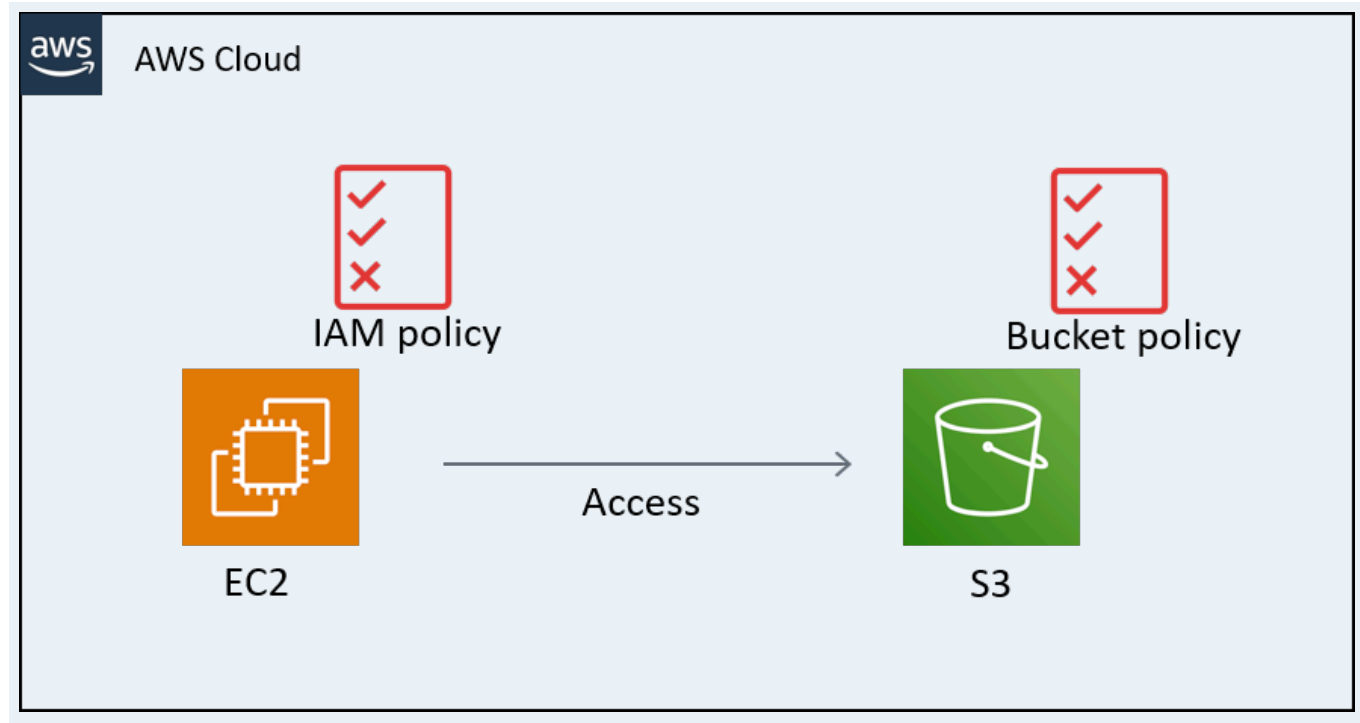
A live index.html is recreatable, but it is normally accessed frequently.

What are Glacier classes for?

Glacier is for information you need to keep but expect to access rarely: old backups, compliance records, historical logs, financial archives or media archives.

3. S3 Bucket Policies and Public Access

A bucket policy is an AWS resource-access policy attached to an S3 bucket. It describes who may perform which actions against which resources.



Ignore EC2 IAM policy for now

Storing something in S3 doesn't automatically mean everyone should be allowed to access it. A bucket **policy defines who is allowed to do what** with the objects in an S3 bucket.

You might want:

- Everyone to be able to **view product images**
- Only your application to **upload new images**
- Only authorized employees to **access invoices**
- Nobody outside your AWS account to **access customer exports**

Policy field	Question it answers	Meaning above
Effect	Allow or deny?	Allow
Principal	Who?	* = anyone/everyone
Action	Do what?	s3:GetObject = read/retrieve an object
Resource	To what?	Every object inside website-bucket

Policy Example

```
{
  "Effect": "Allow",
  "Principal": "*",
  "Action": "s3:GetObject",
  "Resource": "arn:aws:s3:::website-bucket/*"
}
```

KEY IDEA

The policy above means: “Allow anyone to retrieve any object in this bucket.”
It does NOT grant upload, modification or deletion.

Why can public read still be risky?

For a bucket containing only public website assets, public GetObject is intentional.
The risk is that the policy applies to every matching object, including something accidentally added later.

Object	Public GetObject appropriate?
index.html	Yes - it is meant to be delivered to every visitor.
app.js	Yes - browsers need it.
logo.png	Yes - public website asset.
debug-config.json accidentally uploaded	Potentially dangerous - it becomes readable too.
customer-export.csv accidentally uploaded	Dangerous - unintended data exposure.

In production, a common pattern is to keep S3 private and let CloudFront be the public entry point.
The bucket can then grant CloudFront permission to read objects instead of granting the entire internet direct S3 access.

Uploads are a separate permission

Action	Meaning
s3:GetObject	Read/download an object.
s3:PutObject	Upload/write an object.
s3:DeleteObject	Delete an object.

A public GetObject policy does not let anonymous visitors upload files. In a web app with uploads, the backend may upload on the user's behalf or issue a temporary presigned URL after authorizing the user.

4. S3 Encryption

S3 automatically applies server-side encryption to new objects.
With SSE-S3, AWS manages the encryption keys.

Protection	What it protects	Example
Encryption at rest	Data while stored on AWS storage.	SSE-S3 / SSE-KMS
Encryption in transit	Data while moving between systems.	HTTPS/TLS

KEY IDEA

Imagine AWS's underlying physical storage were compromised or stolen.
Encryption at rest means the raw stored bytes are not simply readable as your index.html.

An object can be encrypted at rest and still be publicly readable.
S3 decrypts the object when an authorized GetObject request occurs.
Encryption protects the stored bytes; permissions determine who AWS will return the decrypted object to.

5. CloudFront and Amplify Hosting

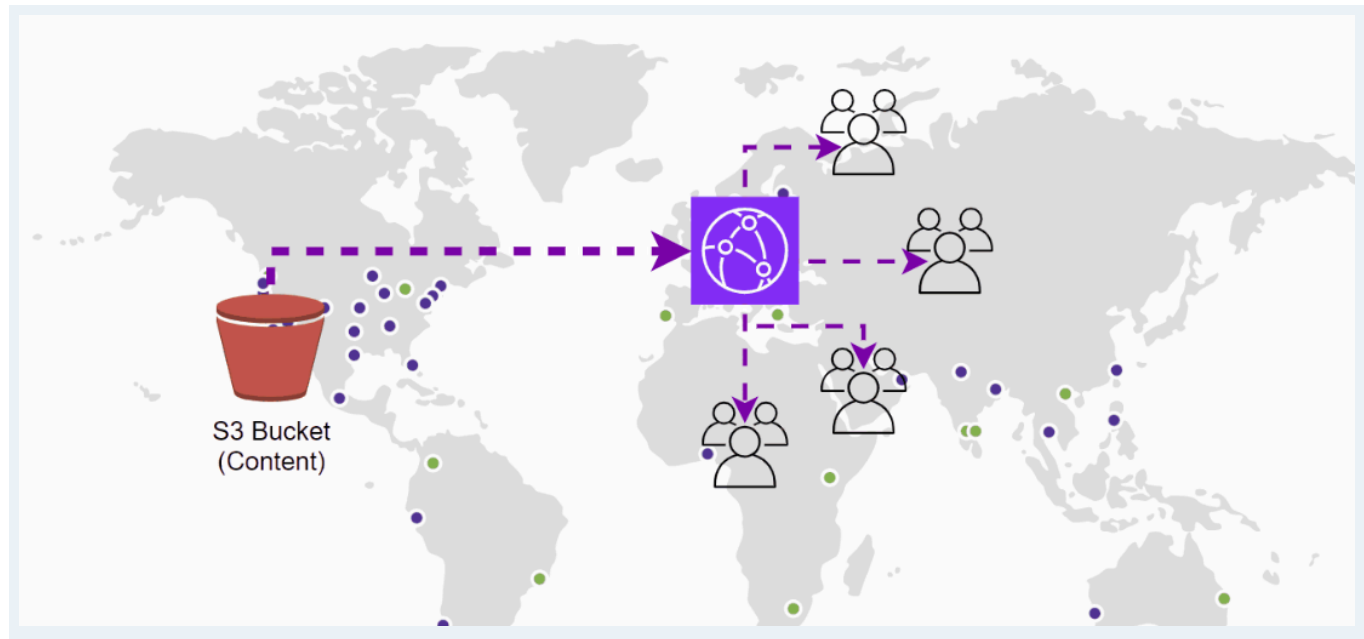
So far, S3 has handled storage and basic static hosting.

The next step is delivery: how do we serve those files efficiently to users, or let AWS manage more of the deployment process for us?

Amazon CloudFront

CloudFront is AWS's content delivery network (CDN).

It caches and delivers content from edge locations closer to users.



S3 stores it; CloudFront delivers it.

AWS Amplify Hosting

Amplify Hosting is a higher-level managed web hosting and deployment experience. It is conceptually much closer to Vercel than manually configuring S3 website hosting.

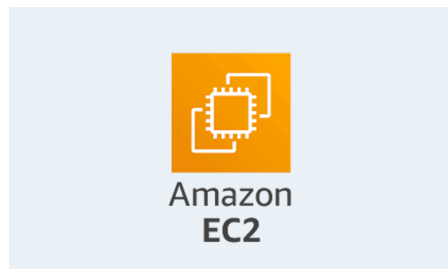
Amplify capability	What it saves you from doing manually
Connect Git repository	Manually moving build artifacts.
Build on Git changes	Running and uploading each deployment yourself.
Automatic deployment	Manual release process.
CDN	Manually configuring content distribution.
HTTPS and custom domains	More infrastructure setup.
Branch/preview environments	Manually maintaining separate deployments.
Redirects and rewrites	Manual routing configuration.

6. Regions, Availability Zones and EC2

Amazon EC2

EC2 stands for Elastic Compute Cloud (E + C²). It is easiest to think of an EC2 instance as a virtual computer rented from AWS.

Concept	Mental model
EC2	The computer: CPU/RAM and an operating system capable of running programs.
EBS	The attachable virtual hard drive/SSD used by that computer.
S3	Object storage: store and retrieve objects rather than use it as the normal OS disk.



How Region, AZ and EC2 fit together

A Region is a geographic AWS area, such as N. Virginia (us-east-1).

Inside a Region are several Availability Zones (AZs).

Each AZ is one or more physically separate data centers.

An EC2 instance runs in one AZ, and an AZ can contain many EC2 instances and other AWS resources.

In short:

Region is the geographic area.

An AZ is an isolated location inside it.

EC2 is the virtual computer running in that AZ.

Why multiple AZs?

Imagine your whole website runs on one EC2 instance. If that instance or its Availability Zone fails, the website goes with it. Running another copy in a different AZ removes that single point of failure.

When adding an EC2 instance: choosing an instance type

The EC2 instance-type list looks enormous because AWS combines workload families, hardware generations, architectures and sizes. Do not start by comparing hundreds of individual names. First choose the family based on what your application needs, then choose the size.

The screenshot shows the AWS Management Console interface for selecting an EC2 instance type. The title is 'Instance types (1/1354+)' with a 'Get advice' link. A search bar is present with the placeholder text 'Find resources by attribute or tag'. Below the search bar is a table of instance types. The table has columns: Instance type, vCPUs, Architecture, Memory (GiB), Storage (GB), Storage type, and Network performance. The 't3.micro' instance type is selected, indicated by a blue circle and a blue highlight. The table lists various instance types including t1.micro, t2.nano, t2.micro, t2.small, t2.medium, t2.large, t2.xlarge, t2.2xlarge, t3.nano, t3.micro, t3.small, and t3.medium. At the bottom right, there are 'Cancel' and 'Select instance type' buttons.

Instance type	vCPUs	Architecture	Memory (GiB)	Storage (GB)	Storage type	Network performance
t1.micro	1	i386, x86_64	0.612	-	-	Very Low
t2.nano	1	i386, x86_64	0.5	-	-	Low to Moderate
t2.micro	1	i386, x86_64	1	-	-	Low to Moderate
t2.small	1	i386, x86_64	2	-	-	Low to Moderate
t2.medium	2	i386, x86_64	4	-	-	Low to Moderate
t2.large	2	x86_64	8	-	-	Low to Moderate
t2.xlarge	4	x86_64	16	-	-	Moderate
t2.2xlarge	8	x86_64	32	-	-	Moderate
t3.nano	2	x86_64	0.5	-	-	Up to 5 Gigabit
t3.micro	2	x86_64	1	-	-	Up to 5 Gigabit
t3.small	2	x86_64	2	-	-	Up to 5 Gigabit
t3.medium	2	x86_64	4	-	-	Up to 5 Gigabit

Family	Optimized for	CPU	RAM	Typical use	Mental shortcut
T	Burstable / light general workloads	Moderate; can burst	Moderate	Small web server, dev/test, low-traffic API	Tiny / occasional CPU
M	General purpose	Balanced	Balanced	Normal backend, application server	Middle / balanced
C	Compute optimized	High	Lower ratio than R	Video encoding, simulations, CPU-heavy processing	Compute / CPU
R	Memory optimized	Moderate-high	High	Large caches, in-memory databases, memory-heavy processing	RAM
I	Storage optimized	Varies	Varies	Workloads needing very fast local storage / I/O	I/O
G / P	Accelerated / GPU	CPU + GPU	Varies	Machine learning, graphics, rendering	GPU

KEY IDEA Pick by bottleneck: light workload -> T; balanced -> M; CPU-heavy -> C; RAM-heavy -> R; storage-I/O-heavy -> I; GPU/accelerated work -> G/P.

Then choose the size

After choosing the family, choose how much of that kind of machine you need. Names such as micro, large, xlarge and 2xlarge mainly represent increasing capacity.

Example	How to read it
t3.micro	T = burstable family 3 = generation micro = small size
c5.large	C = compute optimized 5 = generation large = size
c5.24xlarge	Same compute family/generation, but an extremely large machine

A simple EC2 selection workflow

Question	Decision
Is this just a small/light server with occasional CPU demand?	Start with T
Is the workload balanced with no special bottleneck?	Start with M
Is processing/CPU the bottleneck?	Look at C
Does the application need unusually large amounts of RAM?	Look at R
Is very fast local storage/I/O the main requirement?	Look at I
Does the workload need GPU/accelerators?	Look at G/P
Family chosen - what next?	Choose the smallest size that meets CPU/RAM/network needs, then monitor and resize if necessary.

Example: a small learning or low-traffic backend might use t3.micro. A video-encoding service is more likely to benefit from a C-family instance. A workload holding a very large working set in memory is more likely to benefit from an R-family instance.

Key pair (login)

When launching an EC2 instance, AWS may ask you to select or create a key pair. The key pair is used to prove that you are allowed to log in to the virtual server, commonly through SSH.

Part	Where it is	Purpose
Public key	Stored on the EC2 instance	Lets the server verify a matching private key.
Private key	Kept by you (for example a .pem file)	Used by your SSH client to prove your identity.

Physical analogy: EC2 is a computer in an AWS data center, and the private key is like the key you keep for its locked door. Do not commit the private key to Git, place it in the web project, or expose it publicly.

Should I include a key pair?

Situation	Include one?
Learning / lab and you may want to SSH into the instance	Yes - simplest choice.
You explicitly need direct SSH access	Yes.
Production environment using AWS Systems Manager Session Manager instead of SSH	Not necessarily.
Instance is fully automated and nobody should log in directly	Potentially no; depends on the access design.

KEY IDEA In more mature AWS setups, direct SSH can be avoided by using managed access such as Systems Manager.

Network settings: VPC and subnet

When launching an EC2 instance, you also decide which AWS network the machine belongs to and which smaller section of that network it will be placed in. This is what the VPC and Subnet fields control.

VPC - the private network around your AWS resources

VPC stands for Virtual Private Cloud. Think of it as your own logically isolated private network inside AWS. EC2 instances and other resources placed in the VPC receive addresses from the VPC's private IP address range and can be connected or isolated according to your network design.

Setting from this lab	Meaning
VPC: cloud-first-steps/LabVpc	The pre-built network Cloud Quest expects the EC2 instance to join.
CIDR: 10.0.0.0/16	The private address range reserved for this VPC: 10.0.0.0 through 10.0.255.255 (65,536 total IPv4 addresses).
Auto-assign public IP: Enable	The instance can also receive a public IPv4 address so it can be reached from outside the VPC, subject to routing and firewall rules.

Why 10.0.0.0/16 instead of the default VPC? There is nothing inherently 'better' about 10.0.0.0/16. Cloud Quest already created LabVpc with this address range and built the exercise around it. The instructions therefore require that specific VPC so your instance joins the network containing the lab's expected subnets, routes and other configuration. Choosing the default VPC would put the instance into a different AWS network.

KEY IDEA 10.0.0.0/16 is a private IPv4 range, not a public internet address.

The '/16' describes how large the network is. In this lab we choose it because LabVpc was deliberately configured that way, not because every VPC should use 10.0.0.0/16.

Subnet - a smaller network inside the VPC

A subnet divides the VPC's address space into smaller networks. Importantly, each subnet belongs to exactly one Availability Zone. Choosing a subnet therefore also determines the AZ in which the EC2 instance's network interface is placed.

Lab value	What it tells us
VPC: 10.0.0.0/16	Large parent private network.
Subnet: 10.0.0.0/26	A smaller slice of the VPC's address space.
Availability Zone: us-east-1a	Resources launched into this subnet are placed in that AZ.
Public-SubnetSubnet1	The lab has configured this as its public subnet, with networking intended for internet-reachable resources.

The exercise specifically says to choose the subnet in us-east-1a because this is part of the architecture it is asking you to build. If a second EC2 instance is later required in a separate Availability Zone, you would choose a different subnet that belongs to that other AZ.

Mental model

AWS concept	Physical-world analogy
Region	The city/area where the infrastructure is located.
Availability Zone	A separate building/site within that area.
VPC	Your private company network spanning the Region.
Subnet	A section of that network located in one particular AZ.
EC2	A computer connected to that subnet.
Security group	The firewall attached to the computer/network interface.

Before working with S3, we first need an environment for our application. In this lab, an EC2 instance will act as our web server.

The instance is launched inside the networking environment we covered earlier:

- **Region:** us-east-1
- **VPC:** LabVpc using 10.0.0.0/16
- **Subnet:** 10.0.0.0/26 in us-east-1a
- **EC2 instance:** webserver01

This gives us a running server inside our VPC that we can use as we begin connecting the application to AWS services such as S3.

The screenshot displays the AWS Management Console interface for EC2 instances. At the top, there's a header for 'Instances (1/1)' with a refresh button and buttons for 'Connect', 'Instance state', 'Actions', and 'Launch instances'. Below this is a search bar and a table of instances. The table has columns for Name, Instance ID, Instance state, Instance type, Status check, Availability Zone, and Public IPv4 DNS. One instance, 'webserver01', is listed with ID 'i-0a5b30d6068123bf9', state 'Running', type 't3.micro', and public DNS 'ec2-34-229-241-19'. Below the table, the details for instance 'i-0a5b30d6068123bf9 (webserver01)' are shown. The 'Details' tab is active, displaying the instance summary: Instance ID 'i-0a5b30d6068123bf9', IPv6 address '-', Public IPv4 address '34.229.241.196', Private IPv4 addresses '10.0.0.32', Instance state 'Running', and Public DNS 'ec2-34-229-241-196.compute-1.amazonaws.com'.

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS
webserver01	i-0a5b30d6068123bf9	Running	t3.micro	Initializing	us-east-1a	ec2-34-229-241-19

i-0a5b30d6068123bf9 (webserver01)

Instance summary

Instance ID i-0a5b30d6068123bf9	Public IPv4 address 34.229.241.196 open address	Private IPv4 addresses 10.0.0.32
IPv6 address -	Instance state Running	Public DNS ec2-34-229-241-196.compute-1.amazonaws.com open address

Lab setup: us-east-1 Region, LabVpc (10.0.0.0/16), subnet 10.0.0.0/26 in us-east-1a, then the EC2 instance.

Firewall: Security Groups

A security group acts as a virtual firewall for an EC2 instance. It controls which network traffic is allowed to reach the instance. It controls which incoming network connections are allowed to reach the instance.

Inbound Security Group Rules

▼ Security group rule 1 (TCP, 80, 0.0.0.0/0) Remove

Type [Info](#)
HTTP ▼

Protocol [Info](#)
TCP

Port range [Info](#)
80

Source type [Info](#)
Anywhere ▼

Source [Info](#)
 ×

Description - optional [Info](#)

⚠ Rules with source of 0.0.0.0/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only. ×

Add security group rule

► Advanced network configuration

In this lab, the EC2 instance will act as a public web server. A web browser needs to reach the server on port 80, the standard port for HTTP. We therefore add an inbound HTTP rule.

Setting	Lab value	What it means
Type	HTTP	Allow normal HTTP web traffic.
Protocol	TCP	HTTP uses TCP for this connection.
Port	80	The standard HTTP port.
Source	Anywhere (0.0.0.0/0)	Allow connections to port 80 from any IPv4 address.

Choosing **Anywhere** does not give visitors access to the whole EC2 instance. It only allows the traffic described by this rule. Here, anyone can connect to **port 80 using HTTP** because the website is intended to be public.

Why allow Anywhere?

Anywhere (0.0.0.0/0) means that the rule accepts traffic from any IPv4 address. Whether this is appropriate depends on what the resource is being used for. A public website needs to accept traffic from the internet, while administrative access or databases should usually be restricted to trusted sources.

Example	Appropriate source
Public website on HTTP port 80	Anywhere. The public must be able to reach the website.
SSH administration on port 22	Usually your IP or another trusted network, not the whole internet.
Database port	Usually only the application or trusted internal resources.

Restricting the source reduces unnecessary exposure. If a service only needs to communicate with a specific application, server or administrator, there is little reason to allow the entire internet to attempt a connection. For example, a database used by a web application should normally accept database traffic only from the resources that actually need it. This does not replace authentication or other security measures, but it reduces the number of systems that can even attempt to reach the service.

The warning in AWS is there because Anywhere is a broad permission. It is reasonable for a public web port, but should not be selected automatically for every service. Security group rules should allow only the traffic the application actually needs.

How this fits into the EC2 setup

When launching an EC2 instance, several settings work together to determine **where the server runs, how it connects to the network and how it can be accessed**. None of these settings does everything on its own. A public IP, for example, gives the instance an internet-reachable address, but the security group still determines which incoming traffic is allowed.

The table below brings together the main networking and access settings used when launching our EC2 instance.

Setting	Purpose
VPC	Chooses the private AWS network.
Subnet	Chooses a smaller network and the Availability Zone.
Public IP	Gives the instance an address that can be reached from the internet.
Security group	Controls which types of incoming traffic are allowed.
Key pair	Proves your identity when logging in through SSH.

Together, these settings create the environment around our EC2 instance. The **VPC and subnet** determine where the instance exists within our AWS network, while the **public IP and network routing** make internet communication possible. The **security group** controls which incoming connections are permitted, and the **key pair** can be used to authenticate us when connecting through SSH.

It is useful to think of these as separate layers rather than one big “internet access” setting. An instance can have a public IP without allowing HTTP traffic, or a security group can allow HTTP while the surrounding network has no route to the internet. For the connection to work, the relevant pieces must be configured together.

In this lab, these choices give us what we need: **a web server running inside our VPC that can accept public HTTP traffic while other access remains controlled**.

User data: automatic setup when EC2 starts

User data lets you give an EC2 instance setup instructions before it is launched. For Linux instances this is often a shell script. EC2 runs the script during the initial startup, so common setup work can happen automatically.

User data - optional | [Info](#)
Upload a file with your user data or enter it in the field.



```
#!/bin/bash
sudo yum update -y
sudo yum install -y httpd
sudo yum install -y git
export TOKEN=`curl -X PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-metadata-token-ttl-seconds: 21600"`
export META_INST_ID=`curl http://169.254.169.254/latest/meta-data/instance-id -H "X-aws-ec2-metadata-token: $TOKEN"`
export META_INST_TYPE=`curl http://169.254.169.254/latest/meta-data/instance-type -H "X-aws-ec2-metadata-token: $TOKEN"`
export META_INST_AZ=`curl http://169.254.169.254/latest/meta-data/placement/availability-zone -H "X-aws-ec2-metadata-token: $TOKEN"`
cd /var/www/html
echo "<!DOCTYPE html>" >> index.html
echo "<html lang='en'>" >> index.html
```

☐ User data has already been base64 encoded

In this lab, the script prepares the instance to act as a web server. It updates the operating system, installs Apache and Git, reads information about the EC2 instance, and creates the website files.

Part of the script	Purpose
yum update	Updates installed Linux packages.
yum install httpd	Installs the Apache web server.
yum install git	Installs Git.
169.254.169.254 metadata requests	Reads information about this EC2 instance from the Instance Metadata Service.
instance-id / instance-type / availability-zone	Gets values such as the instance ID, t3.micro, and us-east-1a.
/var/www/html and index.html	Creates content in Apache's website directory.

Why reuse the same user data file?

Imagine setting up the same web server twice by hand, then doing it again for 20 servers. User data lets each new instance start from the same setup instructions, which is faster and keeps the servers consistent.

INFO: Why this is interesting

User data introduces the idea that servers do not have to be configured manually. You can describe the setup once and reuse it when creating new machines. This leads naturally into automation, repeatable deployments, scaling, and Infrastructure as Code.

Ideas this concept leads to

Idea	Why it follows from user data
Repeatable server setup	New instances can receive the same initial configuration.
Scaling	If more servers are needed, they can be created with the same setup instead of configured manually.
Replace rather than repair	A broken server can sometimes be replaced by launching a fresh, consistently configured instance.
Infrastructure as Code	The broader idea is to describe infrastructure and configuration in code instead of relying on manual setup.
Auto Scaling	AWS can create additional EC2 instances when demand increases; automated setup helps those instances become useful without manual work.

For the beginner mental model, treat user data as the EC2 instance's first-start setup instructions. By default, EC2 user data scripts are generally intended for the initial launch rather than every restart.

Adding the second web server

We now create webserver02 as another copy of the same website. It stays in the same Region, but we deliberately place it in a different Availability Zone.

Instance	What it runs	Why it exists
webserver01	The web server and website	First working copy of the application.
webserver02	The same web server and website	A second working copy so the application does not depend on one EC2 instance.

The same user data file is useful here because both instances need the same setup. Each instance can install Apache and create the same website automatically.

Why create a second copy?

Problem with one instance	What the second instance provides
The EC2 instance crashes or becomes unhealthy.	Another copy of the website can still run.
One instance cannot handle enough traffic.	Requests can be shared between multiple instances.
The first server needs maintenance or replacement.	The other server can remain available.

Why another copy? If webserver01 fails, webserver02 can still run. By putting the copies in different AZs, we also avoid losing both servers if one AZ has an outage.

Placement	Result
Two copies in the same AZ	Helps with instance failure and extra capacity, but both depend on the same AZ.
One copy in us-east-1a and one in another AZ	Also protects the application if one Availability Zone becomes unavailable.

Why the same Region? We want separate failure locations without moving the application to another geographic AWS Region. A load balancer can later send traffic to the healthy copies.

Choosing the second Availability Zone

This becomes concrete when choosing the subnet. Both lab subnets are in the same VPC and Region, but each belongs to a different AZ.

The screenshot shows the 'Network settings' section in the AWS console. Under 'VPC - required', the selected VPC is 'vpc-082afe2a57508af4a (cloud-first-steps/LabVpc)' with CIDR '10.0.0.0/16'. Under 'Subnet', two options are listed: 'subnet-089d83a3146940939 cloud-first-steps/LabVpc/Public-SubnetSubnet2' and 'subnet-03b388d7cd893b38b cloud-first-steps/LabVpc/Public-SubnetSubnet1'. The first subnet is selected with a blue checkmark. A search bar is visible above the subnet list. A link 'Create new subnet' is also present.

Subnet	Availability Zone	Use in this lab
Public-SubnetSubnet1	us-east-1a	webserver01
Public-SubnetSubnet2	us-east-1b	webserver02

For webserver02 we choose the subnet in us-east-1b. That keeps it in N. Virginia (us-east-1) while separating it from webserver01 in us-east-1a.

This is the point of the lab: the two servers run the same website, but they do not depend on the same Availability Zone.

7. Elastic Block Store (EBS)

EC2 gives us the virtual computer, but that computer also needs persistent disk storage. EBS is the AWS service that fills that role for EC2.

EBS provides persistent block storage, commonly attached to EC2. The physical analogy is an SSD/hard drive attached to a computer.

EBS volumes belong to an Availability Zone. An EBS volume in one AZ cannot simply be attached directly to an EC2 instance in another AZ.

INFO Keep the storage mental models separate: EBS behaves like a disk attached to EC2; S3 stores independent objects; databases such as RDS/Redshift solve higher-level data problems.

EBS SSD volume types

Type	Name	Best mental model / workload
gp3	General Purpose SSD	Modern default for most normal workloads.
gp2	General Purpose SSD	Older general-purpose generation.
io2	Provisioned IOPS SSD	High-performance, latency-sensitive and critical workloads such as demanding databases.
io1	Provisioned IOPS SSD	Older provisioned-IOPS generation.

IOPS means Input/Output Operations Per Second. Provisioned IOPS volumes are used when you need to specify and depend on a particular level of storage I/O performance.

EBS HDD volume types

EBS also has HDD-backed volume types. They are cheaper for large amounts of data, but are designed around sequential throughput rather than the fast random I/O of SSDs.

Type	Name	Best mental model / workload
st1	Throughput Optimized HDD	Large, frequently accessed data with long sequential reads/writes - e.g. big data, data warehouses and log processing.
sc1	Cold HDD	Large, infrequently accessed data where very low storage cost matters more than performance.

KEY IDEA SSD = fast I/O and low latency. HDD = move large streams of data cheaply. st1 = throughput-focused HDD; sc1 = cold/lowest-cost HDD.

8. Amazon Redshift

Not every database-style workload is about serving an application one request at a time. AWS also provides services designed for analyzing large amounts of historical data.

Redshift is AWS's data warehouse for analytical workloads over large datasets. It supports SQL-style analysis but serves a different purpose from a transactional application database.

Transactional database (e.g. RDS/PostgreSQL)	Redshift data warehouse
Get/update one customer's order.	Analyze years of orders across millions/billions of records.
Many small application operations.	Large analytical queries and aggregations.
Runs the operational application.	Supports analytics, reporting and BI.

KEY IDEA Cloud Practitioner shortcut: Redshift = data warehouse = large-scale analytics using SQL.

9. S3 Select: Concept from the Lab

The lab also introduces a useful data-processing idea: sometimes it is cheaper and faster to retrieve only the part of a stored dataset that you actually need.

The course material referenced S3 Select: a feature that could retrieve only a subset of data from an S3 object using SQL-like expressions rather than transferring the entire object.

Example: instead of retrieving a 5 GB CSV and filtering it in the application, query for only the matching rows and return a much smaller result.

KEY IDEA Remember the concept more than the old feature: query only the data you need instead of moving everything first. S3 Select is legacy for new customers; Amazon Athena is the more relevant AWS service to recognize for SQL queries over data stored in S3.

10. Computing Solutions

Now that we can launch EC2 instances, the next question is how we connect to them, manage them and read information about them.

Connecting with SSH

SSH gives us direct terminal access to an EC2 instance and works well when we only have a few servers. But imagine managing access to 50 instances: keys need to be distributed, protected and removed when access changes. That gets annoying quickly.

AWS Systems Manager

This is where AWS Systems Manager becomes useful. It gives us a central way to manage EC2 instances. Session Manager can provide secure access without opening inbound ports, running a bastion host or managing SSH keys.

Three practical EC2 problems

The module now introduces several features at once. They make more sense if we treat them as answers to three practical questions: how do we access the server, how does the server know about itself, and what do we do when it needs more power?

Problem	AWS concept	Why it matters
How do I securely connect?	EC2 Instance Connect	Uses IAM permissions to control SSH access, reducing the need to distribute and manage long-lived SSH keys.
How does the instance know about itself?	Instance Metadata Service (IMDS)	Lets the instance retrieve details such as its instance ID, type and Availability Zone.
What if the server needs more power?	Change the EC2 instance type	Move to a larger instance type to get more compute capacity. This is vertical scaling.

Instance metadata and the website

We have already used instance metadata in our User Data script. The EC2 instance reads its own metadata, the script places those values into the website, and Apache serves the page to the browser. The browser is displaying the result; it is not directly reading the metadata service.

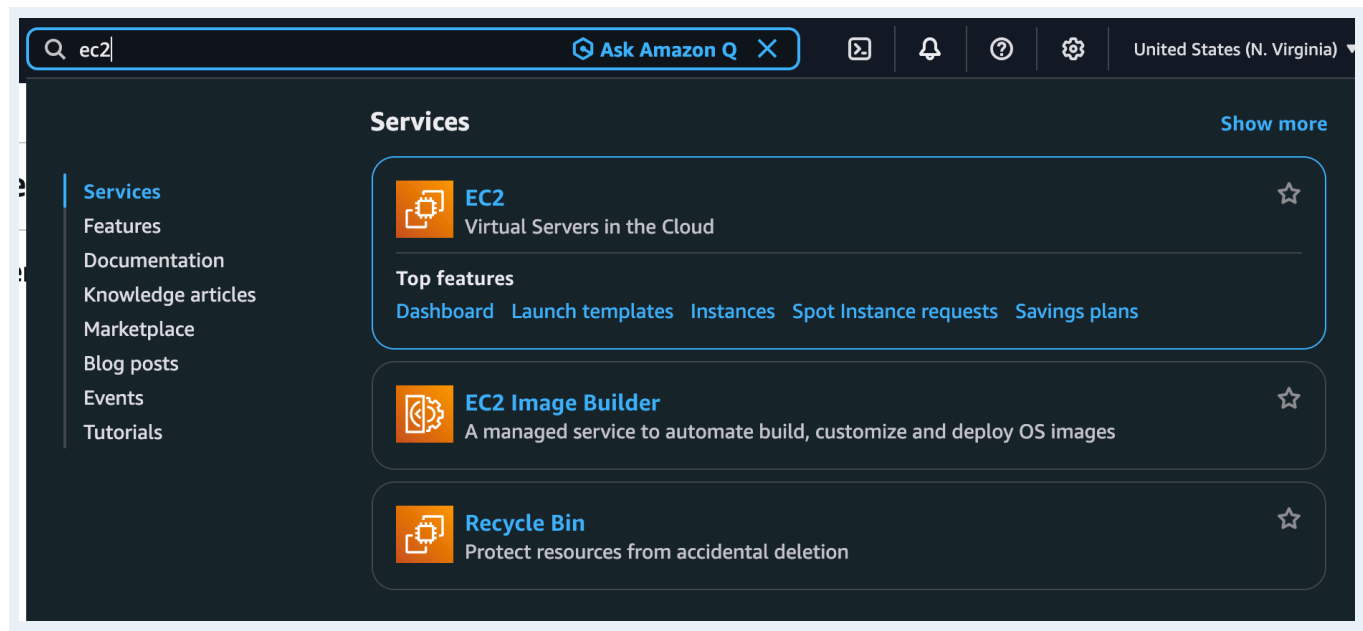
Scaling up versus adding instances

If one EC2 instance is too small, we can give it more capacity by changing to a larger instance type. That is vertical scaling. Earlier we created another EC2 instance instead; adding more instances is horizontal scaling.

Approach	Example	Meaning
Vertical scaling	t3.micro to a larger instance	Make one server more powerful.
Horizontal scaling	webserver01 + webserver02	Add more servers.

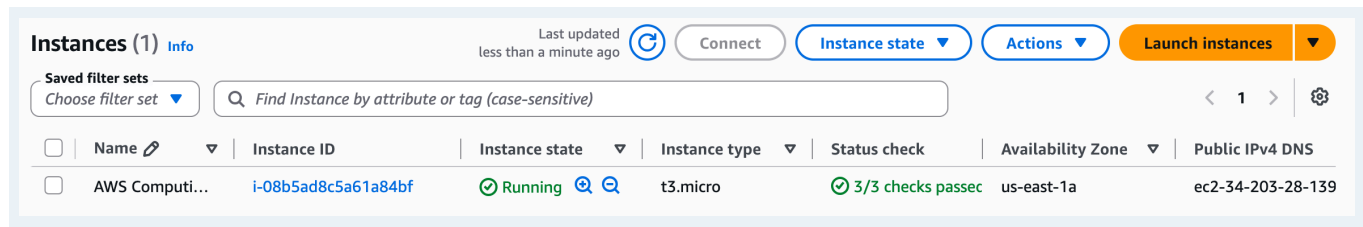
Reviewing and managing an EC2 instance

Once an instance exists, the Console becomes our control panel. We can check what is running, what resources it has, how much alternatives cost, how to connect to it and what happens when we stop or change it.



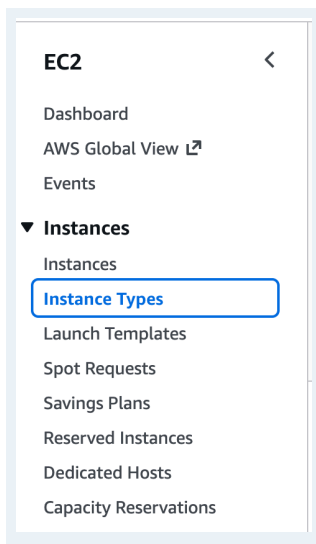
Finding the instance

EC2 → Instances is the quickest overview of a server. It shows whether it is running, its instance type, Availability Zone, status checks and network information. In other words: “What is this machine, and is it healthy?”



Instance type and cost

Instance types decide the CPU, memory and network capacity available to the server. Comparing types also shows an important cloud idea: more resources usually cost more, and different operating-system licenses can change the price. Choose for the workload, not simply the biggest machine.



Instances types: t3.large, r5.large, c5.large			
On-Demand Windows pricing	0.177 USD per Hour	0.218 USD per Hour	0.1108 USD per Hour
On-Demand Linux pricing	0.085 USD per Hour	0.126 USD per Hour	0.0832 USD per Hour
On-Demand SUSE pricing	0.141 USD per Hour	0.182 USD per Hour	0.1395 USD per Hour
On-Demand RHEL pricing	0.114 USD per Hour	0.155 USD per Hour	0.112 USD per Hour
On-Demand Ubuntu Pro pricing	0.089 USD per Hour	0.13 USD per Hour	0.0867 USD per Hour

Connecting to the operating system

The Console manages the EC2 resource, but sometimes we need to work inside Linux itself. The Connect page shows the available access methods. Traditional SSH normally uses a key pair; this lab has no long-lived key pair attached, so AWS-managed options such as EC2 Instance Connect or Session Manager are useful alternatives.

In web browser

In SSH client

Choose how to connect

Public subnet access

Managed SSH keys

EC2 Instance Connect

Use for instances with public IPs or accessible via standard routing. Requires network connectivity to your instance. Connect using temporary SSH keys managed by AWS.

Private subnet access

Managed SSH keys

EC2 Instance Connect Endpoint

Use for instances in private subnets. Traffic is routed through a VPC endpoint. Connect using temporary SSH keys managed by AWS.

Maximum security

IAM authentication

SSM Session Manager

Uses IAM-based authentication. Connect without SSH keys, bastion hosts, or opening any inbound ports.

Direct

Troubleshoot

EC2 Serial Console

Access your instance's serial port for troubleshooting boot, network, and OS issues. Works without network connectivity. Requires account-level enablement.

Settings

Choose public IP address

IPv4 address (34.203.28.139)

IPv6 address

Username

Enter the username defined in the AMI used to launch the instance. If you didn't define a custom username, use the default username, ec2-user. Read your AMI usage instructions to check if the AMI owner has changed the default AMI username.

ec2-user

Cancel

Connect assist

Connect

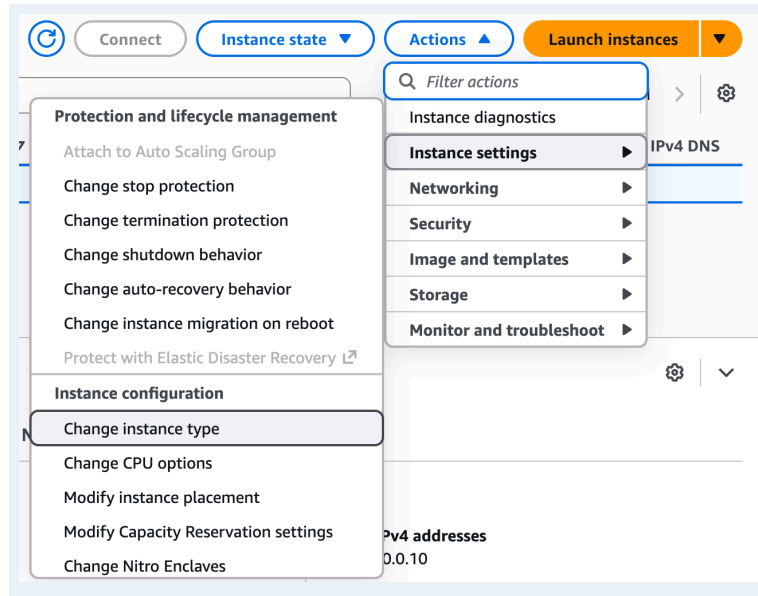
Once connected, normal Linux tools still apply. Commands such as `ls` and `cat` inspect files, while `tail -f` follows a log as new entries are written. In the lab, the application log revealed the Flask/Werkzeug server, its addresses and EC2 metadata such as the instance family, type, ID and Availability Zone.

Changing the instance type in practice

Resizing an EC2 instance is a simple example of vertical scaling: we keep the same instance but give it a different amount of compute capacity. Before changing the instance type, the EC2 instance must be stopped.

Stop the instance before resizing

From the EC2 instance page, choose Instance state → Stop instance. Once it is stopped, Actions → Instance settings → Change instance type becomes available. This means resizing causes downtime for that individual instance, so the change should be planned when availability matters.



Review resources and pricing before applying the change

AWS lets us compare the current instance type with the proposed replacement before saving the change. We can quickly review differences in vCPUs, memory, network performance, architecture and On-Demand pricing.

In this lab, moving from t3.micro to m4.large keeps 2 vCPUs but increases memory from 1 GiB to 8 GiB, while Linux On-Demand pricing rises from \$0.0104 to \$0.1000 per hour.

New instance type		
m4.large		
Only instance types with x86 (32-bit) / (64-bit) architecture can be chosen.		
☒ EBS-optimized EBS-optimized is enabled by default for this instance type		
Instance type comparison		
Attribute	t3.micro	m4.large
On-Demand Linux pricing	0.0104 USD per Hour	0.1000 USD per Hour
On-Demand Windows pricing	0.0196 USD per Hour	0.1920 USD per Hour
vCPUs	2 (1 core)	2 (1 core)
Memory (MiB)	1024	8192
Storage (GB)	-	-
Supported root device types	ebs	ebs
Network performance	Up to 5 Gigabit	Moderate
Architecture	x86_64	x86_64

11. Quick Recall Sheet

Use this as the fast revision page. If a term feels fuzzy, jump back to the chapter where it was introduced.

AWS term	One-line memory hook
S3	Object storage - “store these files/objects for me.”
S3 Standard	Frequently accessed object storage.
One Zone-IA	Rarely accessed + recreatable data in one AZ.
Glacier	Archive data you need to keep but rarely access.
Bucket policy	Who can do what to this S3 resource?
Principal	WHO receives the permission.
GetObject	Read/retrieve an S3 object.
SSE-S3	AWS-managed encryption of S3 objects at rest.
CloudFront	CDN - deliver/cache content near users.
Amplify Hosting	Managed Git-to-web deployment, similar in spirit to Vercel.
Region	Geographic AWS area.
AZ	Isolated AWS location inside a Region.
EC2	Virtual computer/server.
EBS	Persistent virtual disk attached to EC2.
Redshift	Data warehouse for large-scale analytics.